SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA1202

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

*I **P Dinesh Reddy / 192525399** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

A. A smart city surveillance system continuously receives video streams from many cameras. The CPU analyzes the video, the memory stores temporary frames, and the I/O devices transfer data from cameras and storage. During peak hours, a huge amount of data is transferred simultaneously, creating delays.

Analysis of CPU, Memory, and I/O Interaction

CPU: Executes image processing and threat detection algorithms.

- **Memory (RAM):** Temporarily stores video frames and processing data.
- **I/O Devices:** Cameras continuously send video data, while storage devices save recordings.

- When many cameras send data at the same time:
- The CPU waits for data from memory or I/O devices.
- Memory becomes busy handling multiple read/write requests.
- I/O devices generate continuous high-speed data transfers.
- This creates **bus contention**, increasing response time and causing lag in threat detection.

Effect of Bus Structures

- **Single-Bus Structure**
- CPU, memory, and I/O devices share one common bus.
- Only one data transfer can occur at a time.
- High traffic causes bottlenecks and increased waiting time.
- Performance decreases during peak surveillance.

Multi-Bus Structure

- Separate buses are provided for CPU, memory, and I/O communication.
- Multiple data transfers can occur simultaneously.
- Reduces bus congestion and improves data throughput.
- Faster communication results in quicker threat detection and smoother system performance.

Improvements in the Instruction Execution Cycle

The instruction cycle consists of **Fetch → Decode → Execute → Store**.

Performance can be improved by:

- **Instruction pipelining:** Multiple instructions are processed simultaneously, increasing CPU efficiency.
- **Cache memory:** Frequently accessed data is stored closer to the CPU, reducing memory access time.
- **Direct Memory Access (DMA):** I/O devices transfer data directly to memory without continuous CPU involvement.
- **Parallel processing / Multi-core processors:** Multiple video streams are processed at the same time.
- **Efficient scheduling:** Reduces CPU idle time and improves utilization.
- **Conclusion**
- The lag occurs because the CPU, memory, and I/O devices compete for data transfers, especially in a **single-bus architecture**. Using a **multi-bus structure**, **DMA**, **cache memory**, and **instruction pipelining** reduces bottlenecks, improves data transfer speed, and enables faster real-time threat detection in the smart city surveillance system.

2. A real-time stock trading application running on a processor executes millions of instructions per

second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

A. A real-time stock trading application executes millions of instructions every second to process buy and sell orders. During high trading activity, users experience delays because the processor has a **high Cycles Per Instruction (CPI)** caused by frequent memory accesses and branch instructions.

Impact of the Fetch–Decode–Execute Cycle

The CPU processes every instruction through the following stages:

- **Fetch:** The CPU fetches the instruction from memory.
- **Decode:** The instruction is interpreted to determine the required operation.
- **Execute:** The CPU performs the operation and stores the result.

In a stock trading system:

- Frequent **memory accesses** increase the fetch time because the CPU waits for data from RAM.
- Frequent **branch instructions** (e.g., checking trading conditions) may cause branch mispredictions, forcing the CPU to discard partially executed instructions and restart the fetch cycle.
- These delays increase the **CPI**, leading to slower transaction processing and higher response time.

Methods to Reduce CPI and Improve CPU Performance

1. **Cache Memory**
    - Stores frequently accessed trading data closer to the CPU.
    - Reduces memory access time and speeds up instruction fetching.
2. **Instruction Pipelining**
    - Overlaps the fetch, decode, and execute stages of multiple instructions.
    - Increases instruction throughput and reduces execution time.
3. **Branch Prediction**
    - Predicts the outcome of branch instructions before execution.
    - Reduces pipeline stalls and minimizes branch delays.
4. **Out-of-Order Execution**
    - Executes independent instructions while waiting for memory access.
    - Improves CPU utilization and reduces idle time.
5. **Multi-Core Processors**
    - Processes multiple trading requests simultaneously.
    - Improves system throughput during peak trading hours.
6. **Prefetching Techniques**
    - Fetches upcoming instructions and data in advance.
    - Reduces waiting time during instruction execution.

Conclusion

The delays occur because frequent memory accesses and branch instructions increase the **CPI** during the

fetch–decode–execute cycle. Using **cache memory, instruction pipelining, branch prediction, out-of-order execution, prefetching, and multi-core processors** reduces CPI, improves CPU efficiency, and enables faster, real-time transaction processing even under heavy trading loads.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

   A. An industrial temperature control unit uses the **8085 microprocessor** to read temperature values from sensors and control actuators such as heaters and cooling fans. Incorrect temperature readings may occur if the program uses the wrong **addressing mode**, causing the CPU to access incorrect data or memory locations.

Influence of Addressing Modes on Data Access

The 8085 supports different addressing modes, each affecting how data is accessed:

- **Immediate Addressing:** The data is provided directly in the instruction. Suitable for loading fixed values such as threshold temperatures.
- **Register Addressing:** Data is stored in CPU registers, enabling fast processing of sensor values.
- **Direct Addressing:** The instruction specifies the exact memory address where sensor data is stored.
- **Register Indirect Addressing:** A register pair (HL, BC, or DE) holds the memory address of the sensor data, allowing flexible access.
- **Implicit Addressing:** The operand is predefined, usually involving the accumulator.

Possible Causes of Incorrect Temperature Readings

Using the **wrong addressing mode**, causing the CPU to read data from an incorrect memory location.
Incorrect initialization of register pairs (HL, BC, or DE).
Accidental overwriting of sensor data in memory.
Improper use of load/store instructions.
Programming errors in branch or loop instructions that skip important data updates.

Effective Use of the 8085 Instruction Set

- To ensure accurate and reliable operation:
- Use **LDA** and **STA** to correctly load and store sensor values.
- Use **MOV** to transfer data safely between registers.
- Use **MVI** to initialize registers with fixed values.
- Use **CMP** to compare the measured temperature with the desired threshold.

- Use conditional jump instructions (**JZ, JNZ, JC, JNC**) to control heaters or cooling devices based on temperature conditions.
- Use **IN** and **OUT** instructions for reliable communication with input sensors and output actuators.
- Verify register contents before performing calculations and memory access.

Conclusion

The choice of **addressing mode** directly affects how the 8085 accesses sensor data. Incorrect addressing may result in wrong temperature readings and improper actuator control. By selecting the appropriate addressing mode and correctly using **8085 instructions** such as **LDA, STA, MOV, MVI, CMP, IN, OUT, and conditional jumps**, the system can process temperature data accurately and provide reliable industrial temperature control.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand